Tree data structures in Go

Hierarchical structures: binary trees, BSTs, balanced trees, tries, segment trees, Fenwick trees. The Go standard library has almost no tree types, so most of this is hand-rolled.

Binary tree

Each node has up to two children. Useful as the base for BSTs, heaps (conceptually), expression trees, and parse trees.

```
type Tree[T any] struct {
    Value       T
    Left, Right *Tree[T]
}

func (t *Tree[T]) Insert(left, right *Tree[T]) *Tree[T] {
    t.Left = left
    t.Right = right
    return t
}
```

Traversals

```
func (t *Tree[T]) InOrder(visit func(T)) {
    if t == nil { return }
    t.Left.InOrder(visit)
    visit(t.Value)
    t.Right.InOrder(visit)
}

func (t *Tree[T]) PreOrder(visit func(T)) {
    if t == nil { return }
    visit(t.Value)
    t.Left.PreOrder(visit)
    t.Right.PreOrder(visit)
}

func (t *Tree[T]) PostOrder(visit func(T)) {
    if t == nil { return }
    t.Left.PostOrder(visit)
    t.Right.PostOrder(visit)
    visit(t.Value)
}

// level-order (BFS)
func (t *Tree[T]) LevelOrder(visit func(T)) {
    if t == nil { return }
    q := []*Tree[T]{t}
    for len(q) > 0 {
        n := q[0]; q = q[1:]
        visit(n.Value)
        if n.Left  != nil { q = append(q, n.Left) }
        if n.Right != nil { q = append(q, n.Right) }
    }
}
```

Iterative traversal (no recursion stack)

For deep trees, iterative beats recursive for stack safety.

```go
func InOrderIter[T any](root *Tree[T], visit func(T)) {
    stack := []*Tree[T]{}
    n := root
    for n != nil || len(stack) > 0 {
        for n != nil {
            stack = append(stack, n)
            n = n.Left
        }
        n = stack[len(stack)-1]; stack = stack[:len(stack)-1]
        visit(n.Value)
        n = n.Right
    }
}
```

Common uses

- Expression trees (parsers, calculators).
- AST for interpreters and compilers.
- Decision trees.
- Foundation for BSTs, heaps, segment trees.

Binary search tree (BST)

A binary tree where every node's left subtree has smaller values and right subtree has larger values. Insertion, search, and deletion are O(log n) on average, O(n) worst case (degenerate to a list when inserted in order).

```go
type BST[T cmp.Ordered] struct {
    root *bstNode[T]
}

type bstNode[T cmp.Ordered] struct {
    Value       T
    Left, Right *bstNode[T]
}

func (t *BST[T]) Insert(v T) {
    t.root = insert(t.root, v)
}

func insert[T cmp.Ordered](n *bstNode[T], v T) *bstNode[T] {
    if n == nil { return &bstNode[T]{Value: v} }
    switch {
    case v < n.Value:  n.Left  = insert(n.Left,  v)
    case v > n.Value:  n.Right = insert(n.Right, v)
    }
    return n
}

func (t *BST[T]) Contains(v T) bool {
    n := t.root
    for n != nil {
        switch {
        case v == n.Value: return true
        case v <  n.Value: n = n.Left
        default:           n = n.Right
        }
    }
    return false
}

func (t *BST[T]) Delete(v T) {
    t.root = delete(t.root, v)
}

func delete[T cmp.Ordered](n *bstNode[T], v T) *bstNode[T] {
    if n == nil { return nil }
    switch {
    case v < n.Value: n.Left  = delete(n.Left,  v)
    case v > n.Value: n.Right = delete(n.Right, v)
    default:
        // Case: 0 or 1 child
        if n.Left  == nil { return n.Right }
        if n.Right == nil { return n.Left  }
        // Case: 2 children. Find in-order successor (smallest in right subtree).
        succ := n.Right
        for succ.Left != nil { succ = succ.Left }
        n.Value = succ.Value
        n.Right = delete(n.Right, succ.Value)
    }
```

```
    return n
}
```

Properties

- In-order traversal yields sorted values.
- Search: O(log n) balanced; O(n) skewed.
- No guarantee of balance without rotation logic.

For real use, prefer a self-balancing tree (AVL, Red-Black) or `google/btree`. Plain BSTs are educational; the worst case bites in production.

Self-balancing trees

The trees that guarantee O(log n) for all operations by performing rotations after insert/delete.

AVL tree

Each node tracks its height. Difference between left and right subtree heights is kept at most 1. Rotations restore balance after each operation.

Pros: strict balance means slightly faster lookups than Red-Black. Cons: more rotations on insert/delete.

Use cases: in-memory ordered indexes where reads dominate writes. Not in the Go stdlib; use a library if you need one.

Red-Black tree

A relaxed balance: nodes are colored red or black with rules that bound the height to roughly 2*log(n+1). Each operation does at most a constant number of rotations and color flips.

Pros: faster modifications than AVL. Cons: slightly slower lookups.

The Go runtime's internal `runtime/inline` tree and the Linux kernel's process scheduler both use Red-Black. Outside the runtime, `github.com/emirpasic/gods/v2/trees/redblacktree` works.

When to actually use one

In typical Go application code, almost never. Reasons: - `map[K]V` covers most "I need keyed lookup" needs at O(1) expected. - If you need sorted iteration, slice + sort works for small N. - For large sorted in-memory indexes, a sorted slice with binary search wins on cache locality.

The cases that legitimately want a balanced BST: very large in-memory sorted structures with frequent inserts/deletes interleaved with sorted iteration. Most application servers don't have this.

B-tree

Each node holds many keys (often 16-32 or more). Designed for systems where reading a node is expensive (disk, SSD), so you want fewer levels for the same N. Used by: - Databases (Postgres, SQLite, MySQL InnoDB). - Filesystems (BTRFS, NTFS). - In-memory ordered maps where cache locality matters more than asymptotics.

`google/btree` is a well-tuned in-memory generic B-tree:

```
import "github.com/google/btree"

t := btree.NewG[int](32, btree.LessFunc[int](func(a, b int) bool { return a < b }))
t.ReplaceOrInsert(5)
t.Has(5)
```

For most Go application code, plain `map` is still the right answer.

Trie (prefix tree)

A tree keyed by string prefixes. Each path from root to a marked node spells out a stored string. Common uses: autocomplete, IP routing tables, dictionary lookups.

```go
type Trie struct {
    root *trieNode
}

type trieNode struct {
    children map[rune]*trieNode
    isEnd    bool
}

func NewTrie() *Trie {
    return &Trie{root: &trieNode{children: map[rune]*trieNode{}}}
}

func (t *Trie) Insert(word string) {
    n := t.root
    for _, r := range word {
        if _, ok := n.children[r]; !ok {
            n.children[r] = &trieNode{children: map[rune]*trieNode{}}
        }
        n = n.children[r]
    }
    n.isEnd = true
}

func (t *Trie) Contains(word string) bool {
    n := t.findNode(word)
    return n != nil && n.isEnd
}

func (t *Trie) StartsWith(prefix string) bool {
    return t.findNode(prefix) != nil
}

func (t *Trie) findNode(s string) *trieNode {
    n := t.root
    for _, r := range s {
        child, ok := n.children[r]
        if !ok { return nil }
        n = child
    }
    return n
}

func (t *Trie) Suggest(prefix string, limit int) []string {
    n := t.findNode(prefix)
    if n == nil { return nil }
    var out []string
    collect(n, prefix, &out, limit)
    return out
}

func collect(n *trieNode, path string, out *[]string, limit int) {
    if len(*out) >= limit { return }
```

```
    if n.isEnd { *out = append(*out, path) }
    for r, c := range n.children {
        collect(c, path+string(r), out, limit)
        if len(*out) >= limit { return }
    }
}
```

Optimization: array children

For pure-ASCII tries, swap the `map[rune]*trieNode` for a fixed-size array (`[26]*trieNode` or `[128]*trieNode`). Much faster lookup, more memory per node.

Compressed tries (radix tree)

A radix tree compresses chains of single-child nodes into one edge labeled with a string. Saves memory for sparse keys.

Production examples: HTTP routers (`gorilla/mux`, `go-chi/chi`, `httprouter`) use radix trees for path matching. The router itself is essentially `Insert(method+path, handler)` + `Lookup(method+path)`.

Common uses

- Autocomplete (`Suggest(prefix)`).
- Spell check.
- URL routing.
- IP routing (longest prefix match).
- Dictionary of finite words.

Segment tree

A binary tree where each node represents an aggregate over a range of an underlying array. Useful for range queries with point updates in O(log n).

Classic use case: given an array, support: - `update(i, value)` in O(log n). - `query(l, r) = sum/min/max over a[l..r]` in O(log n).

```go
type SegTree struct {
    n    int
    arr []int
    tree []int             // 1-indexed; size 4*n is safe
}

func NewSegTree(arr []int) *SegTree {
    n := len(arr)
    t := &SegTree{n: n, arr: arr, tree: make([]int, 4*n)}
    if n > 0 { t.build(1, 0, n-1) }
    return t
}

func (t *SegTree) build(node, l, r int) {
    if l == r {
        t.tree[node] = t.arr[l]
        return
    }
    m := (l + r) / 2
    t.build(2*node,   l,   m)
    t.build(2*node+1, m+1, r)
    t.tree[node] = t.tree[2*node] + t.tree[2*node+1]
}

func (t *SegTree) Update(i, value int) {
    t.update(1, 0, t.n-1, i, value)
}

func (t *SegTree) update(node, l, r, i, value int) {
    if l == r {
        t.tree[node] = value
        return
    }
    m := (l + r) / 2
    if i <= m { t.update(2*node,   l,   m, i, value)
    } else    { t.update(2*node+1, m+1, r, i, value) }
    t.tree[node] = t.tree[2*node] + t.tree[2*node+1]
}

func (t *SegTree) Query(ql, qr int) int {
    return t.query(1, 0, t.n-1, ql, qr)
}

func (t *SegTree) query(node, l, r, ql, qr int) int {
    if qr < l || r < ql { return 0 }
    if ql <= l && r <= qr { return t.tree[node] }
    m := (l + r) / 2
    return t.query(2*node, l, m, ql, qr) + t.query(2*node+1, m+1, r, ql, qr)
}
```

Variants

- **Lazy propagation** for range updates (mark nodes "needs update" without descending immediately).
- **Min/max segment trees** by replacing `+` with `min` / `max`.
- **Segment tree with custom merge** for non-additive aggregates.

Common uses

- Range sum / min / max queries.
- Inversion counting.
- Skyline / interval problems.
- Sliding window stats with arbitrary aggregates.

Not in the stdlib. Implement when you need it.

Fenwick tree (Binary Indexed Tree, BIT)

A simpler structure than segment tree for "prefix sum with point updates." Smaller code, same O(log n) per op for sums.

```
type BIT struct {
    tree []int   // 1-indexed
}

func NewBIT(n int) *BIT { return &BIT{tree: make([]int, n+1)} }

// Add delta at index i (1-based)
func (b *BIT) Add(i, delta int) {
    for ; i < len(b.tree); i += i & -i {
        b.tree[i] += delta
    }
}

// Prefix sum up to and including i (1-based)
func (b *BIT) Sum(i int) int {
    s := 0
    for ; i > 0; i -= i & -i {
        s += b.tree[i]
    }
    return s
}

// Range sum [l..r] inclusive (1-based)
func (b *BIT) Range(l, r int) int {
    return b.Sum(r) - b.Sum(l-1)
}
```

Properties

- Only supports invertible operations (sum, XOR). Min/max needs a segment tree.
- About half the code of a segment tree.
- Easy to extend to 2D (matrix prefix sums).

Common uses

- Running prefix sums under updates.
- Order statistics (kth element via inversion counting).
- Range counting in coordinate compression.

Tree traversal patterns

Recursive

Clean to write, stack-limited by tree depth. Go's stacks grow as needed, so for balanced trees this is fine even for huge sizes.

Iterative with explicit stack

For very deep skewed trees (a million-deep linked-list-shaped tree), explicit stack avoids runtime stack growth.

Morris traversal

In-order traversal with O(1) extra space by temporarily threading the tree. Useful in memory-constrained environments. Outside competitive programming and embedded code, rarely needed in Go.

Range over function (Go 1.23+)

Wrap a traversal as an iterator:

```go
func (t *Tree[T]) InOrder() iter.Seq[T] {
    return func(yield func(T) bool) {
        var walk func(*Tree[T]) bool
        walk = func(n *Tree[T]) bool {
            if n == nil { return true }
            if !walk(n.Left)  { return false }
            if !yield(n.Value) { return false }
            return walk(n.Right)
        }
        walk(t)
    }
}

for v := range tree.InOrder() { use(v) }
```

The `yield` returning false lets callers break out cleanly.

Tree representation tradeoffs

Pointer-based

```go
type Node struct { Left, Right *Node; Value T }
```

Pros: simple, dynamic, no size limit upfront. Cons: cache-unfriendly; each node is a separate allocation; GC must scan every pointer.

Array-based (heaps, complete binary trees)

```go
parent  := (i - 1) / 2
left    := 2*i + 1
right   := 2*i + 2
```

Pros: cache-friendly, no allocation per node, no pointers for the GC to scan. Cons: assumes complete (or nearly complete) tree. For sparse trees, wasted space.

Index-based (custom arena)

```go
type Node struct { LeftIdx, RightIdx int }
nodes := []Node{}
```

Pros: same cache locality as array-based, but supports sparse trees. Lets you save/load by serializing one slice. Cons: more code; index 0 is no longer “the natural empty”; must agree on a sentinel for nil.

For high-performance trees, arena-based wins on benchmarks by a large margin over pointer-based.

Algorithms on trees (quick survey)

These deserve their own treatment, but the common ones:

Algorithm	What it does	Complexity
BFS	Level-by-level traversal	O(V+E)
DFS	Depth-first traversal	O(V+E)
LCA (lowest common ancestor)	Find shared ancestor of two nodes	O(log n) with preprocessing
Tree DP	Per-node compute based on children	O(n)
Heavy-light decomposition	Path queries on trees	O(log^2 n) per query
Euler tour	Linearize a tree for subtree queries	O(n) prep

Best practices

1. **Default to `map[K]V` for keyed lookup.** O(1) expected vs O(log n) for BST.
2. **For sorted iteration with infrequent updates, use a slice + sort.** Cache locality crushes pointer-chasing BSTs at sizes that fit in L3.
3. **For large in-memory ordered indexes, use `google/btree`.** Don't write your own B-tree.
4. **Tries for prefix queries.** A `map` doesn't do prefix lookups efficiently.
5. **Segment tree when you need range queries with point updates.** Fenwick tree if the aggregate is invertible (sum, XOR).
6. **Prefer iterative traversal for unknown-depth trees.** Recursive blows up on million-deep degenerate trees.
7. **Arena/index-based representation for hot-path trees.** Skip pointers; the GC and CPU will thank you.
8. **Test boundary cases:** empty tree, single node, fully-skewed tree, balanced tree.

Common gotchas

Gotcha	Symptom	Fix
Plain BST degenerates to list on sorted insert	O(n) operations	Use balanced tree or shuffle input
Pointer-heavy tree, slow GC	Long GC pauses	Switch to arena/index representation
Forgetting `if n == nil` base case	Stack overflow / panic	Always check at top of recursive function
Wrong child order on in-order traversal	Reversed output	Left, root, right
Segment tree size too small	Index out of range	Allocate 4*n to be safe
Updating segment tree without propagating up	Stale aggregates	Always update parents after a leaf write
Fenwick tree off-by-one (1-indexed vs 0-indexed)	Wrong sums	Pick a convention and stick with it
Trie memory blow-up for sparse keys	Huge memory use	Use radix tree (compressed trie)
Recursive traversal on million-deep tree	Crash	Iterative with explicit stack
`container/list` for ordered keyed structure	Wrong tool	Use a BST/btree or map+slice
Tree mutated during iteration	Data race / corrupt iteration	Iterate snapshot, or block updates
Self-balancing tree from scratch in production	Bugs in rotations	Use a library

Quick reference

Need	Best Go choice
Keyed lookup (unordered)	`map[K]V`
Keyed lookup (ordered)	`google/btree`
Prefix search	Trie or radix tree
Range sum/min/max + point update	Segment tree
Prefix sum + point update	Fenwick tree (BIT)
Expression / AST	Pointer-based binary tree
Sorted set with iteration	`google/btree` or sorted slice
URL routing	`chi`, `httprouter` (both use radix tree)
In-memory ordered index, very large	`google/btree`
In-memory ordered index, small N	Sorted slice + binary search
Top-K	Heap (see heaps sheet)
Path from root to value	DFS with stack
Level-order processing	BFS with queue
Iterate sorted	In-order traversal
Save/restore	Arena-based with one slice
Range over function	Wrap traversal in `iter.Seq` (Go 1.23+)